Queue Aware Shortest Path Routing

BCSE308P Computer Networks Lab | Final project report

Sashwath J 24BAI0059

Title

Queue-Aware Shortest-Path Routing in a Ten-Node Packet Network: A Comparison with Fixed-Weight Dijkstra

Work done

- Created a directed ten-node, fifteen-link packet network with seven possible S-to-T paths.
- Configured propagation delay, transmission capacity and finite FIFO buffer size for every directed link.
- Implemented fixed-delay Dijkstra and a queue-aware Dijkstra variant with periodic weight refresh.
- Measured peak waiting-queue length during each five-tick update interval; in-flight packets do not count as queue occupancy.
- Used paired random seeds and identical offered packet streams for both methods.
- Ran nine offered loads from 0.5 to 3.5 packets per tick, ten seeds each, for 180 runs per topology.
- Repeated the sweep after increasing the upper corridor capacity, producing 360 final runs in total.
- Recorded per-run and mean delay, loss, throughput, route changes, route selections and packets still in the network.
- Checked packet conservation in every run and generated all graphs from the saved CSV files.

Existing method and the gap

Dijkstra's shortest-path algorithm finds the minimum total cost for nonnegative link weights. In a fixed-weight baseline, the cost is the configured propagation delay. In this network, S-A-D-G-T has delay cost 4, S-B-E-H-T has cost 5, and S-C-F-T has cost 6. The baseline therefore always chooses the upper corridor. Its one-packet-per-tick links become a bottleneck as offered load rises above one packet per tick. Fixed weights do not reveal the waiting queue, so the baseline does not use the other possible paths even when they have spare capacity.

Actual network layout

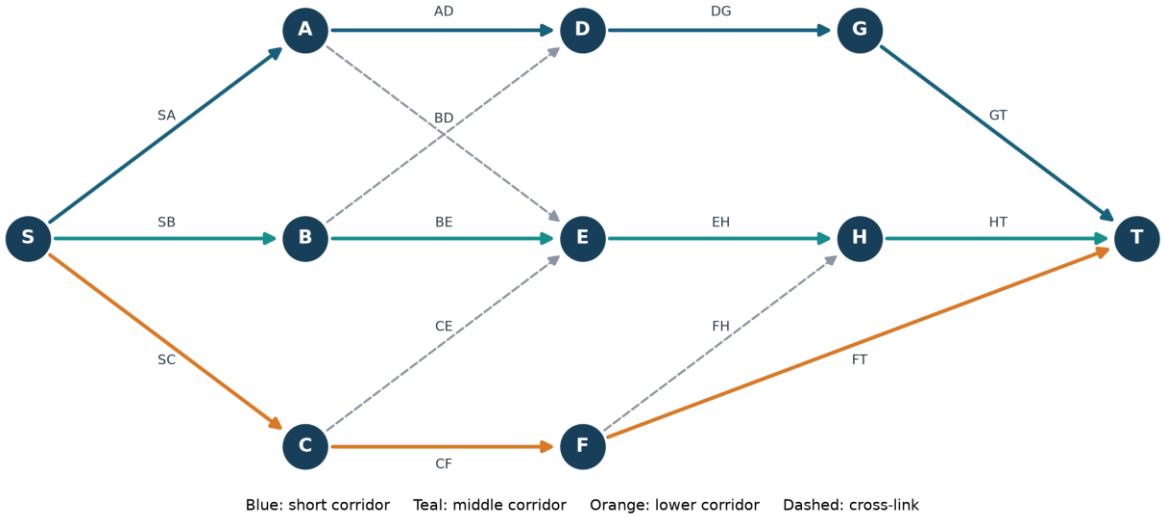


Figure 1. The executable ten-node topology. Arrows show direction. Solid colored links form the three main corridors; four dashed cross-links create additional route choices. Link IDs in this figure match the source code and Table 1.

Link	Direction	Delay ticks	Capacity pkt/tick	FIFO size
SA	S to A	1	1	6
AD	A to D	1	1	6
DG	D to G	1	1	6
GT	G to T	1	1	6
SB	S to B	2	2	8
BE	B to E	1	2	8
EH	E to H	1	2	8
HT	H to T	1	2	8
SC	S to C	2	3	10
CF	C to F	2	3	10
FT	F to T	2	3	10
AE	A to E	3	1	6
BD	B to D	3	1	6
CE	C to E	3	1	6
FH	F to H	3	1	6

Table 1. All fifteen directed link settings in the bottleneck scenario. In the balanced scenario only SA, AD, DG and GT increase from capacity 1 to capacity 2; all other settings remain unchanged.

Seven possible routes

Path	Node sequence	Fixed delay	Path bottleneck
1	S-A-D-G-T	4	1
2	S-A-E-H-T	6	1
3	S-B-E-H-T	5	2
4	S-B-D-G-T	7	1
5	S-C-F-T	6	3
6	S-C-F-H-T	8	1
7	S-C-E-H-T	7	1

Table 2. All seven directed S-to-T paths. The last column is the smallest link capacity on that path, not a guarantee of independent end-to-end capacity where paths share links. Cross-routes can share a bottleneck with a main corridor.

Packet processing and queues

The simulator advances in integer ticks. A newly generated packet is assigned one complete route. Each directed link has a finite FIFO waiting queue and a per-tick service limit. A served packet spends its link propagation delay in flight before arriving at the next node. On arrival, it joins the next link's queue if there is room; otherwise it is dropped. Delivery at T records end-to-end delay as delivery tick minus creation tick. At the end of a run, generated packets equal delivered plus dropped plus packets still waiting or in flight. This conservation condition is asserted in code.

How the Python program is organized

Packet stores creation time, its chosen sequence of link IDs and its current position in that sequence. Link stores source and destination nodes, delay, capacity, buffer size, a FIFO waiting queue and in-flight packets. Network builds the fifteen directed links and adjacency list. shortest_route applies Dijkstra to the current weights and returns a complete S-to-T route. run advances ticks, transfers packets, updates queue observations, generates identical seeded arrivals and counts outcomes. main repeats the runs over loads and seeds, writes per-run and mean CSV files, and plot reads the measured rows to generate the graphs. The source is a single file so the full flow can be demonstrated in the lab.

How the link weight changes

The fixed method uses weight $w = d$. The queue-aware method uses $w = d \times (1 + \alpha q/B)$, where d is link propagation delay, q is the maximum number of packets observed waiting in that link's FIFO queue during the preceding update interval, B is its FIFO size, and $\alpha = 2.5$. Every five ticks the observed peaks are sampled, Dijkstra recomputes a route, and the peaks are reset. Packets arriving in the next interval use that cached route. The maximum over the interval captures short queue bursts that may have cleared at the exact refresh tick. No packet is rerouted after assignment.

Example: SA has delay 1 and buffer size 6. If its observed queue peak is 3, its weight changes from 1 to $1 \times (1 + 2.5 \times 3/6) = 2.25$. If the other upper-route links remain at weight 1, that route totals 5.25, above the middle route's fixed cost 5. The newly computed route can therefore use S-B-E-H-T. A packet never changes the physical capacities; it only changes which route it enters.

Experiment design and metrics

Both algorithms use the same generated arrivals for each seed and offered load. Loads are 0.5, 0.8, 1.0, 1.2, 1.5, 2.0, 2.5, 3.0 and 3.5 packets per tick. Each run lasts 5,000 ticks, and seeds 0–9 are used for each method and load. This gives 2 algorithms × 9 loads × 10 seeds = 180 runs per topology. A balanced-capacity scenario repeats the experiment, making 360 final runs. The reported plots show arithmetic means of the ten per-run measurements at each point.

Average end-to-end delay is calculated for successfully delivered packets only. Loss ratio is dropped divided by generated packets. Delivered throughput is delivered divided by 5,000 ticks. Route changes count switches in route assignments to successive generated packets. Route-selection counts show which of the seven paths carried assigned packets. These definitions matter because a baseline that drops many long-waiting packets can have a deceptively favorable delay among the few packets it delivers.

Outputs

Load	Method	Delay ticks	Loss %	Throughput
0.5	Fixed	5.000	0.000	0.501
0.5	Queue-aware	5.000	0.000	0.501
1.0	Fixed	5.000	0.000	0.999
1.0	Queue-aware	5.000	0.000	0.999
1.2	Fixed	9.983	16.612	0.999
1.2	Queue-aware	5.963	0.002	1.199
1.5	Fixed	9.992	33.318	0.999
1.5	Queue-aware	6.273	0.029	1.499
2.0	Fixed	9.995	49.950	0.999
2.0	Queue-aware	6.833	0.000	1.997
2.5	Fixed	9.996	59.979	0.999
2.5	Queue-aware	7.130	6.619	2.333
3.0	Fixed	9.996	66.633	0.999
3.0	Queue-aware	7.375	11.133	2.663
3.5	Fixed	9.996	71.410	0.999
3.5	Queue-aware	7.703	17.224	2.894

Table 3. Bottleneck scenario means across ten seeds. The raw per-run values and the omitted 0.8 row are in results.csv and summary.csv.

Figure 2. Mean delay across all nine offered loads.

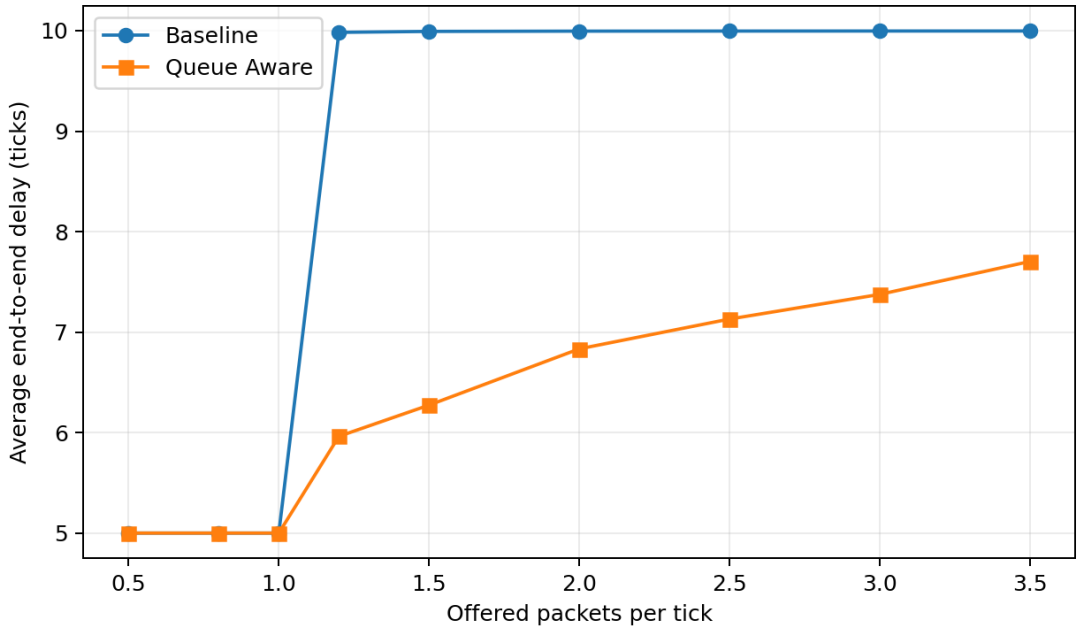


Figure 3. Queue-drop ratio across all nine offered loads.

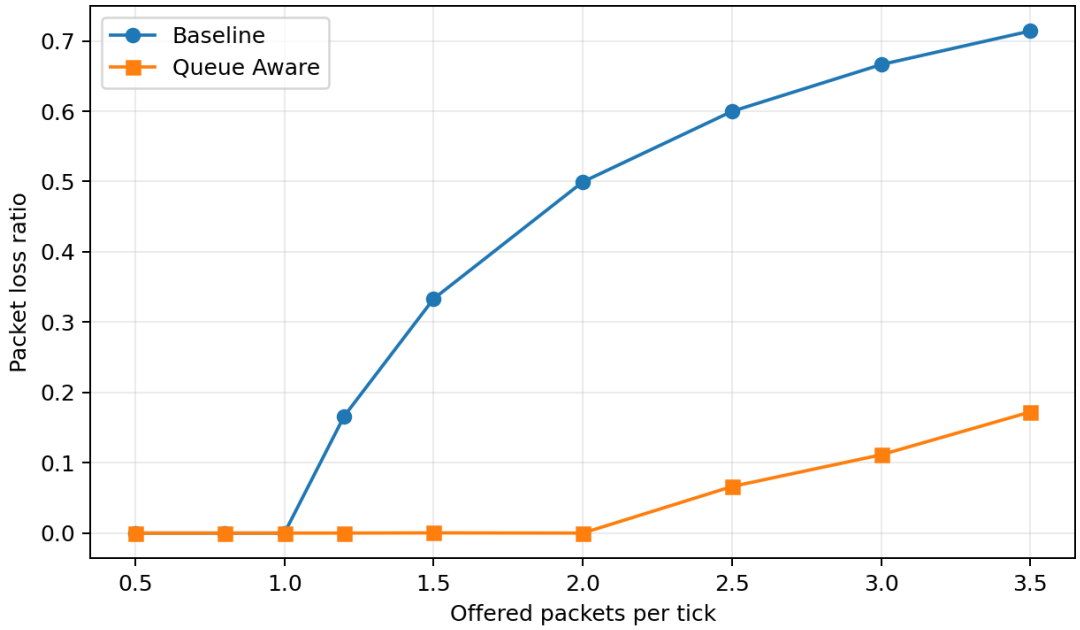
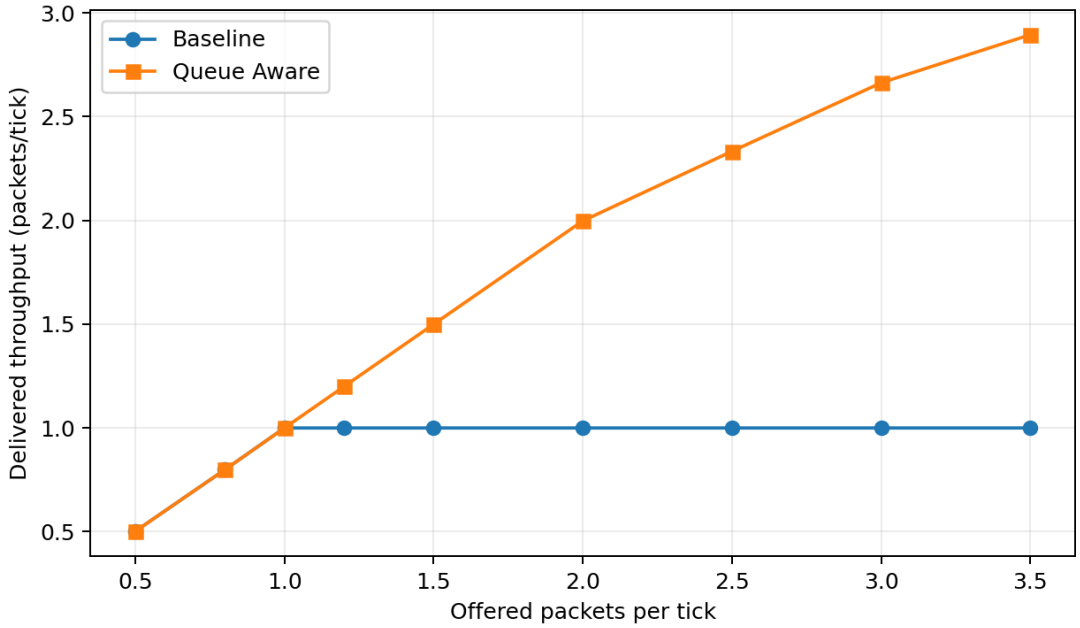


Figure 4. Delivered throughput across all nine offered loads.



Discussion on results

At loads up to 1.0, the two algorithms behave identically because no waiting queue develops on the short corridor. At load 1.2, baseline delay is 9.983 ticks, loss is 16.612%, and throughput is 0.999 packets/tick. Queue-aware delay is 5.963 ticks, loss is approximately 0.0017%, and throughput is 1.199 packets/tick. The route counters show that the queue-aware method assigns about 1,785 packets per run to the middle corridor and a few to the lower corridor, while fixed routing assigns none to either. This supports the mechanism, not only the outcome.

At load 2.0, fixed routing delivers 0.999 packets/tick with 49.95% loss. Queue-aware routing delivers 1.997 packets/tick and records no drops in these ten runs. Its mean route counts are about 3,340 upper, 3,330 middle and 3,330 lower packets per run. The three paths offer spare capacity at this load, explaining why distributing the offered packets improves the result. At load 3.5, queue-aware loss rises to 17.224% and throughput reaches 2.894 packets/tick. Therefore the method does not eliminate congestion at every offered load.

The balanced scenario doubles capacity on the four upper-corridor links. Both algorithms now match at low loads; their difference begins later. At load 3.5, the queue-aware method delivers more packets but may have a larger average delay than the baseline because more delivered packets travel by the longer corridors. The proposed method also changes routes more often and depends on queue observations. Its control-message overhead, stale information, packet sizes, shared physical medium and transport protocols are not modeled. The result is a conditional lab-scale improvement, not an optimality proof or a reproduction of the cited research systems.

Literature connection and references

Dijkstra (1959) supplies the nonnegative-weight shortest-path procedure. RFC 2328 documents the use of Dijkstra in OSPF, providing practical routing context. Shen, Ren and Liu (2016) motivate use of queue information for congestion-aware path selection but use linear prediction and a different mechanism. Bhorkar, Javidi and Snoeren (2012) study congestion-aware routing with queue and link-quality information in a wireless mesh testbed. This simulation is deliberately simpler than those published systems.

E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik 1, 269–271 (1959). DOI: 10.1007/BF01386390.

J. Moy, “OSPF Version 2,” RFC 2328, April 1998. https://www.rfc-editor.org/rfc/rfc2328.html.

Y. Shen, G. Ren and Y. Liu, “Finding the biased-shortest path with minimal congestion in networks via linear-prediction of queue length,” Physica A 452, 229–240 (2016). DOI: 10.1016/j.physa.2016.02.002.

A. A. Bhorkar, T. Javidi and A. C. Snoeren, “Achieving Congestion Diversity in Multi-hop Wireless Mesh Networks,” arXiv:1203.4841 (2012). https://arxiv.org/abs/1203.4841.

GitHub: https://github.com/Sashwath-sash/queue-aware-shortest-path-routing